

# Catalina: In-Storage Processing Acceleration for Scalable Big Data Analytics

Mahdi Torabzadehkashi  
Engineering school, EECS  
University of California, Irvine –  
NGD Systems, Inc  
Irvine, USA  
torabzam@uci.edu

Siavash Rezaei  
Donald Bren School, ICS  
University of California, Irvine  
Irvine, USA  
siavashr@uci.edu

Ali Heydarigorji  
Engineering school, EECS  
University of California, Irvine  
Irvine, USA  
heydari@uci.edu

Hosein Bobarshad  
NGD Systems, Inc  
Irvine, USA  
hossein.bobarshad@ngdsystems.com

Vladimir Alves  
NGD Systems, Inc  
Irvine, USA  
vladimir.alves@ngdsystems.com

Nader Bagherzadeh  
Engineering school, EECS  
University of California, Irvine  
Irvine, USA  
nader@uci.edu

**Abstract**—Cloud applications are increasingly playing a crucial role in big data analytics. New use cases such as autonomous cars and edge computing call for novel approaches mixing heterogeneous computing and machine learning. These applications typically process petabyte-scale datasets, therefore, requiring low-power and scalable storage providing low-latency and high-throughput data access. While data centers have been focusing on migrating from legacy HDDs and SATA SSDs by deploying high-throughput and low-latency NVMe SSDs, the data bottlenecks appear as capacity scales. One approach to tackle this problem is to enable processing to happen within the storage device -in-storage processing (ISP)- eliminating the need to move the data. In this paper, we investigated the deployment of storage units with embedded low-power application processors along with FPGA-based reconfigurable hardware accelerators to address both performance and energy efficiency. To this purpose, we developed a high-capacity solid-state drive (SSD) named Catalina equipped with a quad-core ARM A53 processor running a Linux operating system along with a highly efficient FPGA accelerator for running applications in-place. We evaluated our proposed approach on a case study application for a similarity search library called Faiss.

**Keywords**—Computational storage, In-storage processing, near-data processing, System-on-Chip, FPGA, Accelerator, Big data

## I. INTRODUCTION

The storage industry is facing a challenge in data centers due to the exponential growth of storage capacity demand, changing performance and interface requirements, increasing pressure to reduce power, and the inefficiencies in scaling out storage. The information avalanche leading to the onset of big data and the increasing use of hyperscale server technology have resulted in the creation of an additional storage tier that is quite different from the traditional enterprise storage systems. The main idea behind distributed processing, as it is applied to big data, is to push the processing capability as close to the data as possible to avoid the bottlenecks resulting from the

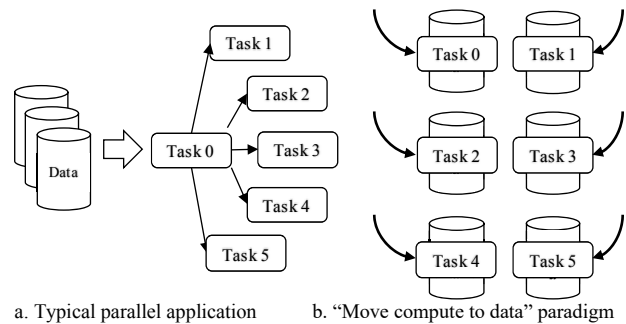


Fig. 1. Parallel processing

need to move data from storage to CPU and its main memory and back. This new paradigm has been successfully applied in widely used data analytics frameworks such as MapReduce and Hadoop (see Fig. 1).

In order to address the three main characteristics associated with big data analytics (Volume, Variety and Velocity) we have developed a novel computational storage solution that combines storage and processing capabilities in one single device.

Hyperscale storage architects must find ways to avoid continuously moving data from storage device to host CPU as well as reducing the power consumed to process data. Webscale data centers have been developing new storage server architectures that favor a significant increase in the capacity (Fig. 2). Evidence of this is highlighted by the storage server platform architecture proposed by Microsoft Azure and Facebook at OpenCompute (OCP) [1] that calls for up to 64 solid state drives motivated by the need for scale up the capacity and consequently the number of drives. This causes a significant bandwidth mismatch from the flash media to the host CPU (in this case 34x) creating the most critical system bottleneck when large data sets are processed (Fig. 2).

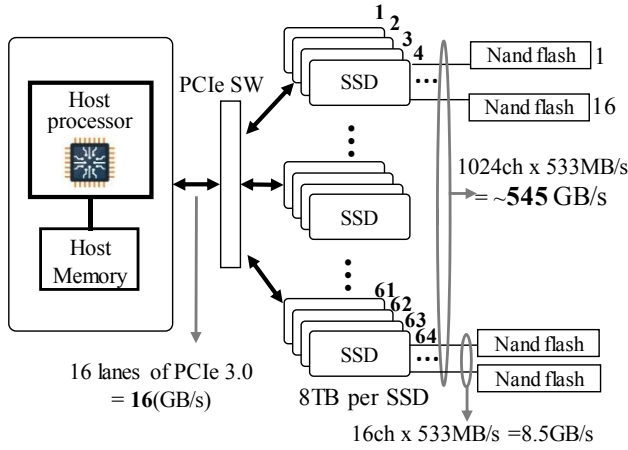


Fig. 2. Bandwidth mismatch in hyperscale servers

One of the key market drivers is the need for making sense, often in near real-time, of the tremendous amount of data continuously stored in the cloud -predicted to reach 44 Zettabytes by 2020, often referred to as big data analytics. There is a wide spectrum of consumers creating this need, from social networks to businesses, law enforcement and intelligence agencies. They all need to consume and analyze enormous amounts of data to make the right business or strategic decisions as for example in the case of image similarity search [2] that will be examined in section IV of this paper.

In this paper, we investigate the use of SoC-based in-storage processing enabled SSD named Catalina powered by a multi-core application processor running a full-fledge 64-bit Linux OS. The operating system provides a flexible environment for running user applications. The programmable logic resources enable the instantiation of hardware accelerators inside the SSD controller delivering flexibility, parallelism and power-efficiency. The programmable accelerators and the application processor, along with the SSD controller, are all implemented on a single SoC which makes it ideal for high-performance data processing inside the storage. For the proof of concept and feasibility of the solution, we have built a storage server with 16 ISP-enabled SSDs. An application can be executed on the application processors and seamlessly make use of the FPGA accelerators thanks to a complete software stack bringing both flexibility and performance.

Overall, the contributions of this paper are summarized as follows:

1. Proposing the architecture of Catalina smart SSD containing an enterprise grade SSD controller with a dedicated quad-core ARM Cortex 64-bit application processor as well as FPGA accelerators all in a single SoC
2. Defining a complete software stack including a 64-bit OS, needed for seamless execution of applications on

distributed processing systems composed of multiple ISP-enabled SSDs

3. Comparing the performance and energy consumption of an image similarity search application to show how ISP-enabled storage devices can augment system performance

The rest of this paper is organized as follows. In Section II, analysis and summary of the related work are provided. Section III describes the architecture of the Catalina as the first SoC-based, flexible, and high-performance ISP-enabled SSD. In addition to the hardware architecture, the section describes a complete software stack. In Section IV, the features of a high-capacity storage server composed of 16 Catalina SSDs for running image similarity search in a distributed manner are described. Then, this section investigates the energy consumption and performance of the proposed solution for running a similarity search application on a large scale data set. Finally, the paper concludes on Section V.

## II. RELATED WORK

A majority of in-storage processing architectures shares the same processing resources for user application and SSD management functions generally implemented in low-level firmware. Example of such ISPs use either ARM processors [3][4][5] or FPGAs as computational units in the storage [6][7]. Although the use of general processors for ISP gives flexibility to the overall design, it does not provide enough processing power for computation intensive tasks, as a user application can interfere with the SSD's performance and vice-versa. For instance, CompStor [5] needs to bundle up 8 ISP-enabled systems to match the processing power of a conventional server with an Intel Xeon processor. Even worse, many of these architectures are based on 32-bit embedded real-time processors that are not suited for executing complex applications or supporting modern operating systems. As an example, Biscuit [8] uses ARM Cortex R7 embedded real-time processors in the storage device. Using real-time processor for two concurrent tasks, user application and SSD firmware control, causes a loss in performance on both tasks. Another example of using ARM for ISP is [3] where Kim et al proposed an ISP system based on ARM9 processors to accelerate specific functions of scan and join operations on databases. Part of these operations is executed on the data path, while data is being prepared for transmission inside the storage. However, this design is intended only for two specific operations and cannot be generalized.

While FPGAs give higher horsepower, a pure FPGA design suffers from the lack of flexibility for different applications [6][9][16]. Jun et al. investigated an architecture for Scalable Multi-Access Flash Store for big data Analytics, called BlueDBM [6][15], where the whole flash controller, client interface and the computation unit are implemented on a single FPGA. By implementing an inter-FPGA communication network, this design can be scaled linearly with increase in number of nodes. However, modifying an FPGA-based system requires lots of time and effort, including RTL design, synthesizing and bitstream creation. Also, as a common problem, all FPGA accelerators must be called and

TABLE 1. RELATED WORK

|                                        | BlueDBM [6]                   | Biscuit [8]                   | Scan & Join [3]                  | Catalina                            |
|----------------------------------------|-------------------------------|-------------------------------|----------------------------------|-------------------------------------|
| ISP engine                             | FPGA<br>(dedicated)           | 2x ARM Cortex R7<br>(shared)  | ARM9<br>(shared)                 | 4x ARM A53 + FPGA<br>(dedicated)    |
| Ease of Programmability                | HARD<br>(RTL design required) | MEDIUM<br>(using Biscuit API) | MEDIUM<br>(application specific) | EASY<br>(e.g. C++, Python)          |
| ISP support file system                | NO                            | NO                            | NO                               | YES<br>(OS support<br>Clustered FS) |
| ISP support for distributed processing | NO                            | NO                            | NO                               | FULL SUPPORT<br>(e.g. MPI / Hadoop) |

commanded by the host system. In some scenarios pre-processing on the host system are required before invoking FPGA accelerators [7]. It can impose too much unnecessary data transfer to the system that has a huge interfere with the core concept of in-storage processing.

Another major challenge is adaptability. In the currently available solutions, data can only be seen as blocks of 0s and 1s; therefore, applications are not dealing with the concept of file systems, as needed for conventional processing. On the other hand, to the best of our knowledge, all the available ISP systems require modifications to the original program to adapt to the ISP architecture or to run a distributed version of the app [4] [8]. For instance, in [4] users are required to break down applications into units of smaller tasks and feed them into ISP. This approach limits the development capability and adds considerable overhead to the overall process. In this we address this issue by deploying an operating system (OS) like Linux on ISP. Running OS inside the storage device enables users to run applications on the storage device with no modification and provides direct access to files.

Another issue is the imposed limitation on data access from the host side during an in storage processing [10]. Example of such system is PRINS [10], an ISP system for resistive content addressable memories (ReCAM). PRINS uses a microcontroller connected to the ReCAM modules to perform in-storage processing. For the sake of data coherency, PRINS does not allow host CPU to access the storage during the in-storage tasks executions.

Table 1 summarizes the main challenges that an efficient and flexible ISP engine faces for Catalina vs. three of the notable previous works. The first row indicates the processing engine for each approach. The second row is related to the adaptability of available applications in high-level programming languages with each architecture. To the best of our knowledge, among all previous works, Catalina is the first approach that provides an ISP that can execute any application with no modification. The third row represents the feature of supporting file system inside ISP. Deploying an operating system in the ISP gives Catalina the capability of running the currently available system for distributed processing like Hadoop and MPI. It allows users to benefit from ISP in distributed processing systems while all the other available ISP architectures fail to support distributed processing in the storage.

### III. CATALINA: SOC-BASED ISP-ENABLED SSD

In the previous section, we discussed the challenges against the design of a flexible and high-performance ISP-enabled storage system and how other researches addressed some challenges but could not address both flexibility and performance challenges together. This section argues that the key to address all of the challenges is a SoC-based ISP-enabled SSD where all the SSD controller, application processor, and FPGA accelerator are embedded in a single chip. Additionally, porting a full-fledge OS inside SSD provides the necessary tools for making a parallel and distributed processing environment for user tasks. This feature is crucial in data centers where thousands of servers containing tens of thousands of storage units process data to deliver service to customers.

An operating system can provide all the tools needed for running distributed tasks. Also, it supports different types of file systems; thus, dealing with the concept of files inside the SSD will be possible. This section divides the architecture of Catalina into four subsections: hardware architecture, software stack, case study description, and the FPGA accelerator architecture.

#### A. Catalina Hardware

The architecture of Catalina is shown in Fig. 3. We have used a Xilinx Zynq Ultrascale + MPSoC [11] as a platform. This FPGA is composed of two subsystems: a processing system (PS) and a programmable logic (PL). Zynq devices feature a quad-core ARM® Cortex-A53 platform running up to 1.5GHz. Combined with dual-core Cortex-R5 real-time

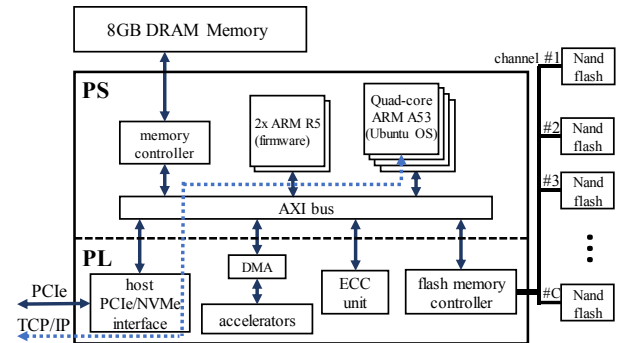


Fig. 3. Catalina hardware architecture

processors, a Mali-400 MP2 graphics processing unit, and 16nm FinFET+ programmable logic [12]. Both PL and PS parts are packaged in a single chip so data movement between them benefits from high-bandwidth, low-latency, and power-efficient interchip AXI data links.

The current implementation of Catalina SSD supports up to 24TB flash storage. This is an enterprise grade solid state drive using 64 layers 3D TLC NAND flash compliant with PCIe and NVMe protocols. The SSD datapath (comprised of modules such as PCIe and NVMe protocol controllers, LDPC error correction engine, flash controller, and flash phy) is implemented using the PL side's reconfigurable logic. The programmable flash memory controller can manage up to 16 flash channels connected to the NAND flash devices that make up the storage media. Each flash channel can provide 533 MB/s data bandwidth per the NAND device specification. As a consequence, a processing engine inside the SSD controller can access a very high-bandwidth, low-latency data bus. The control path is partly implemented in the hardware and partly in the firmware. This relatively complex firmware is executed in the two ARM Cortex R5 real-time processors instantiated in the PS side of the Zynq MPSoC. Contrary to some architectures proposed in the literature, Catalina dedicates the 64-bit quad-core ARM Cortex A53 application processor subsystem exclusively to user applications and supports a state-of-the-art 64-bit Linux operating system. Additionally, the architecture includes FPGA accelerators that can be seamlessly tapped for executing performance-critical elements of the user's applications.

In our platform, the two processor subsystems have access to 8GB of DRAM and communicate using a simple shared mailbox scheme. When a user application uses FPGA accelerator, the SSD controller subsystem takes care of moving user data from storage media to DRAM and writing any output data resulting from data processing to the storage media. FPGA accelerators use a DMA to access the DRAM without the intervention of the OS running on the ARM A53 cores. The hardware features of both subsystems of Catalina is presented

TABLE 2. HARDWARE FEATURES OF CATALINA

|                |                   |                                                     |
|----------------|-------------------|-----------------------------------------------------|
| SSD Controller | DRAM Memory       | 8 GB                                                |
|                | Processing engine | 2x ARM R5 cores @ 600MHz<br>FPGA soft IPs @250MHz   |
|                | Host interface    | NVMe over PCIe Gen3                                 |
| ISP-engine     | DRAM Memory       | 8GB                                                 |
|                | Processing engine | Quad-core ARM A53 @1GHz<br>FPGA Accelerator @250MHz |
|                | Host interface    | TCP/IP over PCIe                                    |

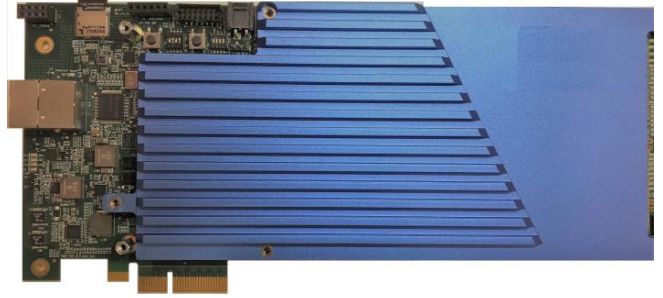


Fig. 4. Catalina Prototype

in Table 3.

The ISP engine is able to communicate with the host using TCP/IP connection tunneled through PCIe. The implementation details of the TCP/IP connection over NVMe/PCIe and also the SSD management software and hardware, e.g. flash translation layer, garbage collection, NVMe/PCIe host interface etc., are outside of the scope of this paper.

The 64-bit operating system combined with the TCP/IP connection through NVMe/PCIe gives Catalina the demanding features for supporting distributed processing tools such as MPI and Hadoop MapReduce. We have shown in our experiment result section that multiple Catalina SSDs can be orchestrated to run distributed computing algorithms in-storage efficiently without moving data to the host. Fig. 4 shows Catalina, a PCIe full-height/three-quarter-length edge card.

### B. Catalina Software stack

The complex hardware of Catalina is not useful without a complete software stack to provide a seamless programming model. The software stack provides the communication

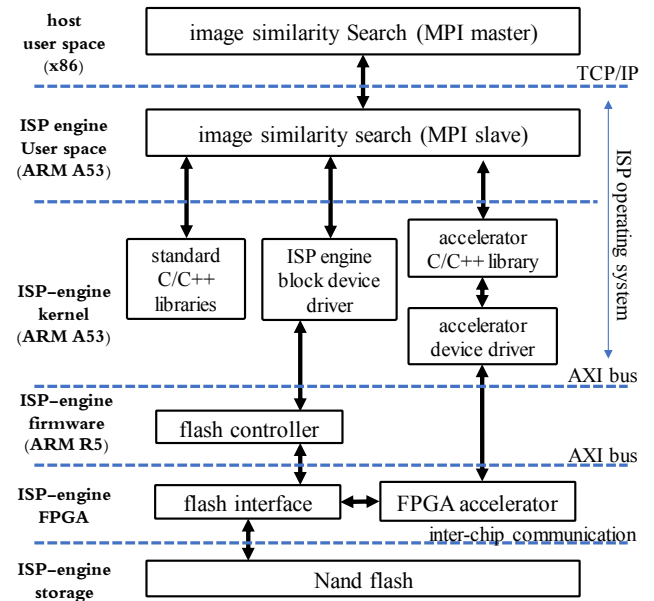


Fig. 5. Catalina software stack



mechanism between the SSD controller subsystem and ISP engine, so the (storage) device's operating system can mount a partition in the storage media. Additionally, the user application running in the ISP engine should be able to offload a task to the FPGA accelerator. Beside all aforementioned essential mechanisms, the software stack should orchestrate multiple Catalina SSDs to enable distributed processing.

Fig. 5 provides a detail description of the software stack. User data (files) is stored in the storage media in the lowest level in the stack. These files are only accessible through the SSD controller subsystem and no other component in Catalina can access them directly. The ISP-engine communicates with the SSD controller subsystem for flash memory read/write operations and mounting the storage space. This access is provided by a block device driver in the kernel of the storage device's OS. The SSD controller sends data directly to the FPGA accelerator. In other words, when a user application offloads a task to the FPGA accelerator, the input data does not need to be written to the DRAM memory and then be sent to the FPGA accelerator. Instead, the FPGA reads the data directly through the SSD controller. The result could be written back to the SSD or may be sent to the DRAM memory based on the user preference.

The FPGA accelerator is a module attached to the AXI bus

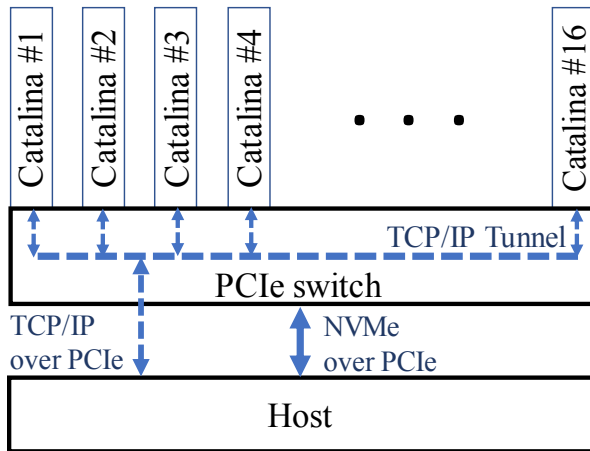


Fig. 6. Storage server using 16 Catalina SSDs

and needs a device driver inside the ISP-engine operating system; so, the user application can offload tasks to it. On top of the device driver, there is a C/C++ library for the accelerator providing high-level APIs for user application.

Other standard libraries are available in the embedded operating system of the ISP engine. However, due to the existence of the ISP engine operating system, the new packages and libraries for different languages like Python and Java can be easily added to the OS. Overall, the user application running inside the SSD can access the storage media via the ISP engine block device driver. Above all of the aforementioned layers, there is a layer providing tools for distributed processing task.

As stated before, Catalina supports a TCP/IP connection over PCIe link using tunneling. Each Catalina in a system (or server) can obtain an IP address via this link and connect to any TCP/IP network. This way, multiple Catalina SSDs on a network can communicate with each other, and such a network is the infrastructure of the distributed processing tools such as MPI. Host server plays the role of master in the MPI-based distributed environment on top of multiple Catalina SSDs. Fig. 6 shows a server with 16 Catalina boards that form an MPI-based distributed environment to run the experiments in this paper. In the server shown in the Fig. 6, the image similarity search application is executed in a distributed manner on both host server cores and the Catalina SSDs ISP engines.

### C. Case study description

For the evaluation of the proposed framework we used an open-source case study library called Faiss [13] which is originally developed by Facebook. Faiss is a library for image similarity search, i.e., allows users to search for multimedia documents that are similar to each other. In Faiss each multimedia item (E.g. image, sound, or video) is represented as a vector. Two vectors are defined as similar when they are close in the Euclidean space. Finding the Euclidean distance is usually composed of an inner product of two vectors describing two items. Since the vectors are defined in high-dimensional spaces and each dimension is defined as a floating-point number, so each member of the dataset is a tuple composed of at least 128 floating-point numbers. For computing the distance between these high-dimensional vectors, the inner product between them should be calculated which is a heavy task especially when this comparison should be made in very large datasets. In this paper we focus on FPGA-based acceleration of the inner production of query vectors and the dataset.

Faiss includes different algorithms for the purpose of similarity search like Flat, IVF, and Product Quantization (PQ). The flat is a brute-force algorithm that searches the entire dataset to find the exact matches. The other algorithms search over only a part of the dataset that makes them faster comparing to Flat algorithm in the price of accuracy. However, regardless of the deployed algorithm, the computationally intensive part of all of them is the inner product of vectors. The FPGA accelerator developed in Catalina provides APIs to do the inner product of very large floating-point vectors.

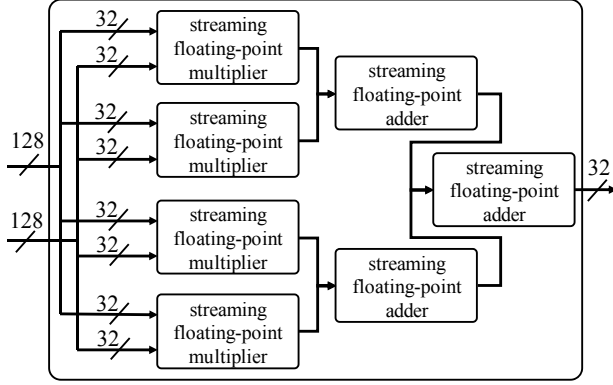


Fig. 7. floating-point vector multiplier

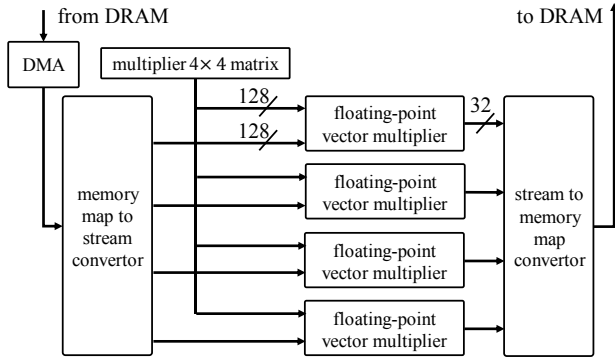


Fig. 8. FPGA accelerator

#### D. FPGA Accelerator hardware description

Our FPGA accelerator multiplies a 4 by  $N$  single-precision floating-point matrix to a 4 by 4 matrix in a pipelined design, where  $N$  can be any integer. In our case study application,  $N$  is a large number-in the order of million. The main module of the accelerator is a vector multiplier that calculates the inner product of two 4D floating point vectors. The architecture of this module called *floating-point vector multiplier* is shown in the Fig. 7.

This module depicted in Fig.7 is replicated 4 times in the FPGA accelerator (Fig.8). A DMA transfers data from storage device's DRAM to a *memory map to stream convertor* module. The floating-point multipliers are AXI stream interfaced, thus, it is possible to feed the accelerator at a very high data rate. Finally, the streaming data is converted to AXI memory map format, and the result is directly written back to the DRAM.

This accelerator on the Zynq Ultrascale+ MPSoC consumes 7% of the CLBs, 3.5% of LUTs, and less than 3% of DSP blocks. The accelerator operates at 250 MHz clock frequency providing up to 3.8 GFLOPs.

## IV. EVALUATION

In this section, we evaluate the effect of the FPGA accelerator in the context of ISP-enabled storage system. We investigate the processing power a set of ISP-enabled SSDs augments to a host system. In the following subsection, the experimental setup will be described and we discuss the architecture of our testbed which is a Xeon server attached to multiple Catalina SSDs.

#### A. Experimental Setup

The testbed is a server with 16 Catalina SSDs forming a 384TB storage system as shown in Fig. 6. This server setup includes a 20-core Xeon 2630 v4 processor running at 2.2GHz with 512GB of DDR4 memory. The Catalina SSDs connected to the server CPUs via a PCIe switch and mounted as NVMe drives each providing 24TB storage space. The Catalina SSDs connected to the server CPUs via a PCIe switch and mounted as NVMe drives each providing 24TB storage space. The second communication mechanism used by the in-storage processing system is a TCP/IP link created using tunneling over PCIe as shown in Fig. 6. The network created thanks to TCP/IP links to each Catalina drive is the backbone of the MPI-based processing system running on the storage server and 16 ISP engines of Catalina SSDs.

In order to implement the image similarity search, we used the Faiss library for executing the compute intensive task of finding similar images to a given query image over a data set with one billion elements. The test application reported in this paper searches for 100 similar items in the entire data set thus requiring at least 1 billion inner product of two 128 bits floating-point vectors. In this experiment, we used a brute-force search algorithm (Flat) from the Faiss library to search the ANN\_SIFT1B dataset [14]. The brute-force algorithm was chosen in order to create a baseline to be used in the different configurations described in the remainder of this section. We did not explore other algorithms that trade-off search accuracy for performance because it does not fall within the scope of this paper. For each experiment, two subsets of this dataset are used, namely database and query. Table 3 shows the detail of this dataset.

The ANN\_SIFT1B is a very large dataset that we distributed evenly between the Catalina SSDs. The master MPI image similarity search based on the Faiss library runs on the host server. While some of the image similarity searches run on the host, each query is dispatched to all the Catalina SSDs that perform the search function in parallel on their corresponding

TABLE 3. TEST DATASET DETAILS

| Test Dataset      | ANN_SIFT1B           |
|-------------------|----------------------|
| Dimension         | 128                  |
| database set size | 1,000,000,000 images |
| query set size    | 10,000 images        |

portion of the dataset. Upon completion each of the local results is sent to the host and then MPI master takes care of aggregating these results and generating the final top 100 similar items for every query.

### B. Experimental Results

This section provides a performance and power comparison of running the Faiss similarity search application on seven different platforms:

1. the host with 16 ISP-disabled Catalina SSDs (Catalina is used only as storage).
2. 6 Catalina SSDs with ISP enabled (software only - using the ARM Cortex A53 processors) augmenting the processing power of the host server and executing the search in parallel using the data stored locally. In this case no HW acceleration is enabled
3. Similar to the second case, but the Catalina SSDs utilize the FPGA accelerator
4. 8 Catalina SSDs are used with ISP enabled but no FPGA acceleration.
5. 8 Catalina SSDs are used with ISP enabled with FPGA acceleration.

6. 16 Catalina SSDs are used with ISP enabled but no FPGA acceleration.
7. 16 Catalina SSDs are used with ISP enabled with FPGA acceleration.

The diagram in Fig. 9 shows as we increase the number of Catalina SSDs with ISP-enabled, the performance of the application increases. We achieved an 11X improvement in performance when 16 Catalina SSDs are used to offload computation from the host CPU. This diagram also shows the performance gain when using the FPGA acceleration in the ISP-enabled storage system. When Catalina SSDs use FPGA acceleration there is a 3X gain over the same experiment running on host and ISP processors only, i.e. software only.

The power consumption was measured over the processing of a total of 10,000 image queries fed in a rate that saturates the system. The energy consumption per query was then derived as considering an average over the total duration of the experiment. The experiment was performed in a similar way for all 7 aforementioned cases.

Fig.10 shows how the heterogeneous computing approach results in a 7X reduction in the energy per query going from 825J per query when the application runs exclusively on the host CPU down to 119.13J per query when using the host CPU combined with the 16 sets of ARM Cortex A53 processors and FPGA accelerators.

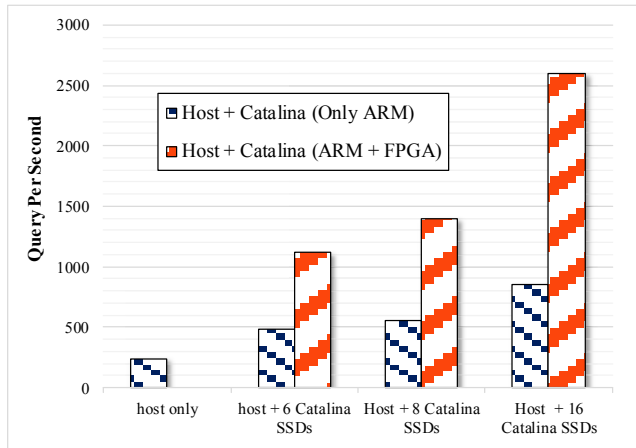


Fig. 9. Performance experimental results

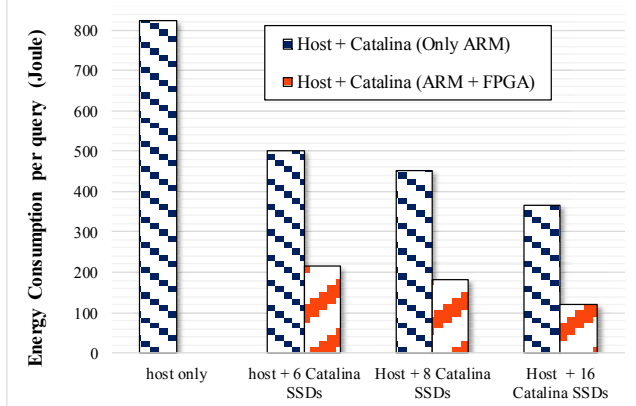


Fig. 10. Energy consumption experimental results

### V. CONCLUSION

In this paper we have investigated an in-storage processing system and demonstrated that it can be effectively used to augment the performance of conventional computing systems. We designed Catalina, an enterprise grade SSD with in-storage processing capability, using a dedicated 64-bit application processor running a Linux operating system. These computing resources address the requirements to run complex applications in-place and deliver additional performance while reducing energy consumption. The elaborate software stack enables a seamless programming model that is essential to port complex applications and facilitate the orchestration of multiple drives in a distributed processing environment based on MPI. In addition to a dedicated embedded processor, we accommodate a heterogeneous computing system by deploying an FPGA-based acceleration. The experimental results showed significant performance improvement and energy reduction when combining a heterogeneous in-storage processing architecture with traditional host-based processing. The results also highlight that in-storage processing scales naturally as data grows which makes it an attractive architecture for big data analytics.

### REFERENCES

- [1] "Server/ProjectOlympus," [online], Available: <https://www.opencompute.org/wiki/Server/ProjectOlympus>
- [2] J. Johnson, M. Douze, and H. Jegou. "Billion-scale similarity search with GPUs," arXiv:1702.08734, 2017.

- [3] S. Kim, H. Oh, C. Park, S. Cho, S. W. Lee, B. Moon, "In-storage processing of database scans and joins," *Information Sciences: An International Journal*, pp. 183-200, January 2016.
- [4] Y. Kang, Y. Kee, E. L. Miller and C. Park, "Enabling cost-effective data processing with smart SSD," 2013 IEEE 29th Symposium on Mass Storage Systems and Technologies (MSST), Long Beach, CA, 2013, pp. 1-12.
- [5] M. Torabzadehkashi, S. Rezaei, V. Alves and N. Bagherzadeh, "CompStor: An In-storage Computation Platform for Scalable Distributed Processing," 2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), Vancouver, BC, 2018, pp. 1260-1267.
- [6] S.-W. Jun, M. Liu, S. Lee, J. Hicks, J. Ankcorn, M. King, S. Xu, and Arvind, "BlueDBM: An appliance for big data analytics," In *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA)*, New York, NY, USA, pp. 1-13, ACM, 2015.
- [7] S. Rezaei et al. "Scalable Multi-Queue Data Transfer Scheme for FPGA-based Multi-Accelerators," In *proceedings of 36th IEEE International Conference on Computer Design (ICCD)*, 2018.
- [8] B. Gu, A. Yoon, D. Bae, I. Jo, J. Lee, J. Yoon, J. Kang, M. Kwon, C. Yoon, S. Cho, J. Jeong, and D. Chang. "Biscuit: a framework for near-data processing of big data workloads," In *Proceedings of ISCA*, 2016.
- [9] X. Song, T. Xie and W. Pan, "RISP: A Reconfigurable In-Storage Processing Framework with Energy-Awareness," 2018 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID), Washington, DC, 2018, pp. 193-202.
- [10] R. Kaplan, L. Yavits and R. Ginosar, "PRINS: Processing-in-Storage Acceleration of Machine Learning," in *IEEE Transactions on Nanotechnology*, vol. 17, no. 5, pp. 889-896, Sept. 2018.
- [11] "SoCs, MPSoCs & RFSocS," [online]. Available: <https://www.xilinx.com/products/silicon-devices/soc.html>
- [12] "Zynq UltraScale+ MPSoC," [Online]. Available: <https://www.xilinx.com/products/silicon-devices/soc/zynq-ultrascale-mpsoc.html>
- [13] "Faiss," [online]. Available: <https://github.com/facebookresearch/faiss>
- [14] "Datasets for approximate nearest neighbor search," [online]. Available: <http://corpus-texmex.irisa.fr>
- [15] S.-W. Jun, M. Liu, K. E. Fleming, Arvind, "Scalable multi-access flash store for big data analytics," In *Proceedings of the 2014 ACM/SIGDA International Symposium on Field-programmable Gate Arrays*, pp. 55-64, 2014.
- [16] S. Rezaei et al., "Data-rate-aware FPGA-based acceleration framework for streaming applications," 2016 International Conference on ReConfigurable Computing and FPGAs (ReConFig), Cancun, 2016, pp. 1-6.